

TEST PLAN FOR COURSEHUB

ChangeLog

Version	Change Date	By	Description
version number	Date of Change	Name of person who made changes	Description of the changes made
1	2nd March 2026	Opeyemi Ogundimu	Initial Testing plan
2	26th March 2026	Opeyemi Ogundimu	Initial mutation testing plan aligned with the Sprint 2 Test Plan

1 Introduction

1.1 Scope

CourseHub is a course platform project. In Sprint 2, the implemented feature is User Authentication & Role Management (Professor/Student), including registration, login, JWT/session handling, and protected routes.

Functional Scope

- **User Authentication & Role Management:** Users can register with name, email, password, and role (Professor/Student), then log in with valid credentials.

- **Credential Security Flow:** Passwords are hashed before storage; successful authentication establishes user access state and redirects to dashboard.
- **Role-Aware Access:** Role is captured at sign-up and used to enforce role-specific access behavior (e.g., restricted professor-only areas).
- **Error Handling in Auth Flows:** Registration/login return clear user-facing errors (e.g., duplicate email, invalid credentials, invalid payload).
- **Course Creation & Management (new feature scope):** Professors can create, edit, and delete their own courses; students cannot perform professor-only course management actions.
- **Course CRUD UX Expectations:** Required-field validation, success feedback, redirect behavior, delete confirmation, and immediate UI state update after deletion.
- **Mutation target scope:** the focus of this document is backend service-layer logic exercised through JUnit 5 unit tests.
- **Mutation execution scope:** feature-specific PIT runs may target selected service classes such as RegisterService, AuthService, and CourseService, depending on the feature being analyzed.
- **Stretch (deferred):** Course material upload and material categorization.

Non-Functional Scope

- **Security:** Password hashing, authenticated session handling, and role-based authorization checks for protected actions.
- **Data Integrity:** Ownership checks on course updates/deletes and safe handling of dependent records during deletion.
- **Validation & Correctness:** Server/client validation for required fields and consistent HTTP status/error responses.
- **Usability:** Clear forms, understandable error messages, and predictable redirect/navigation behavior after auth and course actions.
- **Reliability:** Core auth and course workflows are covered by automated tests and must pass CI before merge/release. Mutation results must be reproducible when re-run locally and in CI for the selected feature scope.
- **Maintainability:** Feature behavior is implemented through modular backend services/controllers and testable frontend components. Mutation testing will be kept focused on feature-level service classes so that surviving mutants are easier to analyze and fix.
- **Test Strength:** mutation testing will be used to confirm that the unit tests detect meaningful code changes rather than only achieving line coverage.
- **Fault Detection:** the team will use mutation testing to expose weak assertions, missing branch coverage, and insufficient negative-path tests.
- **Release Readiness:** mutation analysis is intended to strengthen confidence in the backend services before merge and release.

1.2 Roles and Responsibilities

Name	Net ID	GitHub username	Role
Pankaj Jhanji	jhanjip	fruitlesspeak	Full-Stack Developer
Jessie Ttito Tello	ttitotej	jessiettito	Full-Stack Developer
Mohamed Hammam	hammamm	mhammam2	Full-Stack Developer
Opeyemi Ogundimu	ogundimo	ogundimo	Full-Stack Developer

Full-Stack Developers:

Roles:

- Build end-to-end features from UI to API to database.
- Ensure features are secure, tested, and releasable through CI.

Responsibilities:

- Build and maintain end-to-end features across Vue frontend, Spring Boot APIs, and PostgreSQL data models (including auth, validation, and role-based behavior).
- Ensure release quality by writing/fixing automated tests, debugging cross-layer issues, and keeping CI/CD checks passing before merge.

2 Test Methodology

CORE FEATURE 1: User Authentication & Role Management

Secure sign-up/login functionality supporting distinct roles (Professor/Student) to control access to specific features.

Unit Tests

A) Registration Logic (Service) {2 tests}

1. `registerWithValidPayloadSavesUserAndReturnsResponse`: verifies valid registration normalizes input, hashes password, saves user, and returns expected response fields.
2. `registerWithExistingEmailThrowsConflictException`: verifies duplicate email throws `EmailAlreadyInUseException` and skips password encoding.

B) Login/Auth Logic (Service) {6 tests}

1. `loginWithValidCredentialsCreatesSessionAndReturnsDashboard`: verifies valid login creates session attributes and returns correct role/dashboard path.
2. `loginWithUnknownEmailThrowsInvalidCredentials`: verifies unknown email throws `InvalidCredentialsException` and no session is created.
3. `loginWithWrongPasswordThrowsInvalidCredentials`: verifies bad password throws `InvalidCredentialsException` and no session is created.
4. `getCurrentSessionWhenAuthenticatedReturnsProfile`: verifies existing session returns populated current-session profile.
5. `getCurrentSessionWithoutSessionReturnsEmpty`: verifies no session returns empty result.
6. `logoutInvalidatesSession`: verifies logout invalidates active session.

C) Registration/Login API (Controller) {8 tests}

1. `registerWithValidPayloadReturns201AndResponseBody`: verifies POST `/api/auth/register` success returns 201 and created user body.
2. `registerWithDuplicateEmailReturns409`: verifies duplicate registration returns 409 with conflict message.
3. `loginWithValidPayloadReturns200AndResponseBody`: verifies POST `/api/auth/login` success returns 200 with `userId`, `role`, `dashboardPath`.
4. `loginWithInvalidCredentialsReturns401`: verifies invalid credentials return 401 and error message.

5. `loginWithInvalidPayloadReturns422`: verifies invalid login payload validation returns 422 with field errors.
6. `sessionWhenAuthenticatedReturns200`: verifies `GET /api/auth/session` returns authenticated session data.
7. `sessionWhenUnauthenticatedReturns401`: verifies `GET /api/auth/session` returns 401 when not authenticated.
8. `logoutReturns204`: verifies `POST /api/auth/logout` returns 204.

D) Error Handling {3 tests}

1. `handleInvalidCredentialsReturnsUnauthorizedMessage`: verifies auth exception handler returns invalid-credentials message.
2. `handleValidationErrorReturnsBadRequestMessage`: verifies auth validation handler returns invalid-payload message.
3. `handleEmailConflictReturnsConflictMessage`: verifies API exception handler returns 409 + "Email already in use" payload.

Acceptance Tests

1. As a guest user, I want to register with email/password so that I can create a CourseHub account.
 - Given I'm in the role of a guest user
 - I open the CourseHub registration page (`/register`)
 - Then the system shows the registration form with required fields: Full Name, Email, Password, Confirm Password, and Role (Student/Professor)
 - When I enter a valid full name
 - And I enter a valid email address
 - And I enter a valid password
 - And I confirm the same password
 - And I select a role (Student or Professor)
 - And I click the Create Account button
 - Then the system should create a new account for me
 - And the system should proceed to sign me in and take me to my dashboard (or show sign-in prompt if auto sign-in is unavailable)
2. As a registered user, I want to log in so that I can access my specific dashboard.
 - Given I'm in the role of a registered user
 - I open the CourseHub login page (`/login`)
 - Then the system shows the login form with required fields: Email and Password
 - When I enter my registered email
 - And I enter my correct password
 - And I click the Sign In button

- Then the system should authenticate my account
- And the system should redirect me to my dashboard (/dashboard/{userId})

Mutation Tests

A) Registration Logic (Service) {3 tests}

1. `mutation_registerWithValidPayloadSavesUserAndReturnsResponse`: verifies valid registration normalizes input, hashes password, saves the user, assigns the expected role, and returns the expected response fields.
2. `mutation_registerWithExistingEmailThrowsConflictException`: verifies duplicate email registration throws `EmailAlreadyInUseException`, skips password encoding, and prevents persistence.
3. `mutation_registerWithProfessorRoleSavesProfessorAndReturnsProfessorResponse`: verifies professor registration stores the professor role correctly, normalizes the email, hashes the password, and returns the expected professor response data.

B) Login/Auth Logic (Service) {9 tests}

1. `mutation_loginWithValidStudentCredentialsCreatesSessionAndReturnsStudentDashboard`: verifies valid student login creates the session, stores the correct session attributes, and returns the expected student dashboard path.
2. `mutation_loginWithValidProfessorCredentialsCreatesSessionAndReturnsProfessorDashboard`: verifies valid professor login creates the session and returns the expected professor dashboard path.
3. `mutation_loginWithUnknownEmailThrowsInvalidCredentials`: verifies unknown email login throws `InvalidCredentialsException` and does not create a session.
4. `mutation_loginWithWrongPasswordThrowsInvalidCredentials`: verifies invalid password login throws `InvalidCredentialsException` and does not authenticate the user.
5. `mutation_getCurrentSessionWhenAuthenticatedAsStudentReturnsProfile`: verifies an authenticated student session returns the expected current-session profile and dashboard path.
6. `mutation_getCurrentSessionWhenAuthenticatedAsProfessorReturnsProfessorDashboard`: verifies an authenticated professor session returns the expected role-aware dashboard path.
7. `mutation_getCurrentSessionWithInvalidSessionUserIdReturnsEmptyAndInvalidatesSession`: verifies an invalid session user id is handled safely by invalidating the session and returning an empty result.

8. `mutation_getCurrentSessionWithoutSessionReturnsEmpty`: verifies the service returns an empty result when no HTTP session exists.
9. `mutation_logoutInvalidatesSession`: verifies logout invalidates the active session and kills mutants that skip session invalidation.

CORE FEATURE 2: Course Creation & Management:

A set of tools for Professors to create, manage, and delete courses.

Unit Tests

A) Course Creation & Management Logic (Service) {10 tests}

1. `createWithHttpsLinkSavesLinkAsIs`: verifies valid `https://` link is preserved on create.
2. `createWithWwwLinkNormalizesToHttps`: verifies `www...` link is normalized to `https://www...`
3. `createWithHttpLinkThrowsValidationError`: verifies `http://` link is rejected and course is not saved.
4. `createWithMalformedLinkThrowsValidationError`: verifies malformed link is rejected and course is not saved.
5. `createWithBlankLinkStoresNull`: verifies blank link is normalized to null.
6. `createWithNonProfessorThrowsError`: verifies non-professor/invalid professor id is rejected and no save occurs.
7. `updateCourseAsOwnerUpdatesEditableFieldsAndReturnsResponse`: verifies owner can update editable fields (description, tags, material, dueDate, link) and gets updated response.
8. `updateCourseAsNonOwnerThrowsForbiddenAndDoesNotSave`: verifies non-owner professor update is forbidden and no save occurs.
9. `deleteCourseAsOwnerDeletesCourse`: verifies owner can delete course.
10. `deleteCourseAsNonOwnerThrowsForbiddenAndDoesNotDelete`: verifies non-owner professor delete is forbidden and no delete occurs.

B) Course Creation & Management API (Controller) {12 tests}

1. `createCourseAsProfessorReturns201AndResponse`: verifies POST `/api/courses` by professor returns 201 with created course.
2. `createCourseAsStudentReturns403`: verifies student create attempt returns 403.
3. `createCourseWithoutSessionReturns401`: verifies unauthenticated create returns 401.
4. `createCourseWithMissingRoleReturns401`: verifies missing session role returns 401.

5. createCourseWithMissingUserIdReturns401: verifies missing session user id returns 401.
6. createCourseWithBlankTitleReturns422: verifies blank title fails validation (422).
7. createCourseWithBlankCodeReturns422: verifies blank code fails validation (422).
8. updateAsOwnerProfessorReturns200AndResponseBody: verifies owner professor PATCH /api/courses/{uuid} returns 200 with updated body.
9. updateAsNonOwnerReturns403: verifies non-owner professor patch returns 403 with ownership error.
10. updateAsStudentReturns403WithoutCallingService: verifies student patch returns 403 and service is not called.
11. deleteAsOwnerProfessorReturns204: verifies owner professor delete returns 204.
12. deleteWithoutSessionReturns401WithoutCallingService: verifies unauthenticated delete returns 401 and service is not called.

Acceptance Tests

1. **Professor can create a course and it appears in catalog**
 - Given I am logged in as a professor
 - When I fill in required course fields and click **Create**
 - Then the course is created successfully
 - And the new course appears in the course catalog/list
 - And a student account can view it and enroll
2. **Professor can edit course details they created**
 - Given I am logged in as the professor who created a course
 - And I am viewing that course
 - When I click **Edit**
 - And I update description, tags, material, and due date
 - And I save changes
 - Then the updated values are persisted
 - And the course displays the new description, tags, material, and due date
3. **Professor can delete a course they created**
 - Given I am logged in as the professor who created a course
 - And I am viewing that course
 - When I click **Delete**
 - And I confirm the delete action
 - Then the course is permanently removed from the platform
 - And it no longer appears in the professor dashboard/catalog
 - And it cannot be retrieved by direct course URL/API lookup

CORE FEATURE 3: Enrollment & Discovery

Unit Tests

A) Enrollment Lifecycle Logic (Service) {14 tests}

1. `enrollCreatesNewActiveEnrollmentWithStudentAndCourseIds`: verifies a new enrollment stores the correct student id and course id and is persisted as an active enrollment.
2. `reactivateInactiveEnrollmentMarksItActiveAndSaves`: verifies an existing inactive enrollment is reactivated and saved rather than duplicated.
3. `enrollWhenAlreadyActiveThrowsDuplicateErrorAndDoesNotSave`: verifies duplicate active enrollment attempts throw an error and do not save a new or changed enrollment.
4. `deactivateActiveEnrollmentMarksItInactiveAndSaves`: verifies unenrollment deactivates an active enrollment and persists the updated state.
5. `deactivateWhenAlreadyInactiveThrowsAndDoesNotSave`: verifies deactivation fails for an already inactive enrollment and does not save.
6. `deactivateWhenEnrollmentMissingThrowsNotFound`: verifies deactivation throws the expected not-found exception when no enrollment record exists.
7. `enrollWhenStudentMissingThrowsNotFound`: verifies enrollment fails when the student account does not exist.
8. `enrollWhenProfessorIdPassedThrowsNoStudentFound`: verifies a professor account cannot be treated as an eligible student enrollment target.
9. `enrollWhenCourseMissingThrowsNotFound`: verifies enrollment fails when the target course does not exist.
10. `findByUserReturnsActiveEnrollments`: verifies student-specific enrollment lookup returns the expected active enrollments.
11. `findByCourseReturnsActiveEnrollments`: verifies course-specific enrollment lookup returns the expected active enrollments.
12. `isEnrolledReturnsTrueForActiveEnrollment`: verifies active enrollment status is reported as true.
13. `isEnrolledReturnsFalseForInactiveEnrollment`: verifies inactive enrollments are not treated as active access.
14. `isEnrolledReturnsFalseWhenEnrollmentMissing`: verifies missing enrollment records return false.

Mutation Tests

A) Course Creation & Management Logic (Service) {20 tests}

1. `mutation_createWithHttpsLinkSavesLinkAsIs`: verifies a valid `https://` course link is preserved exactly during creation.

2. `mutation_createWithWwwLinkNormalizesToHttps`: verifies a `www.` course link is normalized to `https://...` before save and response mapping.
3. `mutation_createWithHttpLinkThrowsValidationError`: verifies insecure `http://` links are rejected and the course is not saved.
4. `mutation_createWithMalformedLinkThrowsValidationError`: verifies malformed links are rejected and the course is not saved.
5. `mutation_createWithBlankLinkStoresNull`: verifies a blank link is normalized to `null`.
6. `mutation_createWithMissingProfessorThrowsError`: verifies course creation fails when the professor does not exist.
7. `mutation_createWithExistingNonProfessorUserThrowsError`: verifies a non-professor user cannot create a course and the save path is blocked.
8. `mutation_createWithBlankOptionalTextFieldsStoresNulls`: verifies blank optional text fields are normalized to `null`.
9. `mutation_createWithNullOptionalFieldsStoresNulls`: verifies null optional fields remain null throughout create and response mapping.
10. `mutation_findAllReturnsMappedCourses`: verifies `findAll()` returns mapped course response objects rather than an empty or unmapped result.
11. `mutation_findByUuidReturnsMappedCourse`: verifies `findByUuid()` returns the expected mapped course response.
12. `mutation_findByUuidWhenCourseMissingThrowsNotFound`: verifies a missing UUID throws the expected not-found exception.
13. `mutation_findByProfessorReturnsMappedCourses`: verifies professor-specific retrieval returns the correct mapped courses.
14. `mutation_searchReturnsMappedCourses`: verifies search returns the expected filtered and mapped course results.
15. `mutation_updateCourseAsOwnerUpdatesEditableFieldsAndReturnsResponse`: verifies the course owner can update editable fields and receives the correct updated response.
16. `mutation_updateCourseAsOwnerUpdatesTitleAndCode`: verifies the owner can update course title and code successfully.
17. `mutation_updateCourseAsOwnerBlankOptionalFieldsBecomeNull`: verifies blank optional fields are normalized to `null` during update operations.
18. `mutation_updateCourseAsNonOwnerThrowsForbiddenAndDoesNotSave`: verifies a non-owner professor cannot update another professor's course and no save occurs.
19. `mutation_deleteCourseAsOwnerDeletesCourse`: verifies the owner can delete the course successfully.

20. `mutation_deleteCourseAsNonOwnerThrowsForbiddenAndDoesNotDelete`: verifies a non-owner professor cannot delete another professor's course and no delete occurs.

Acceptance Tests

Acceptance testing for this feature was performed by end-users or team members acting as student users to confirm that the enrollment and discovery requirements were met through normal UI usage.

1. Student can browse available courses in the catalog
 - Given I am logged in as a student
 - When I navigate to the course catalog page
 - Then I can see a list of available courses
 - And each course shows basic information such as title and code
 - And each course has an option to enroll if I am not already enrolled
2. Student can search for courses by tag
 - Given I am logged in as a student and viewing the course catalog
 - When I enter a tag such as **Java** in the search/filter field and submit
 - Then only courses containing that tag are displayed
 - And courses without that tag are not shown in the filtered results
3. Student can enroll in a course from the catalog
 - Given I am logged in as a student and viewing the course catalog
 - When I click **Enroll** on a course I am not yet enrolled in
 - Then the enrollment is completed successfully
 - And I am added to the student list for that course
 - And the course is marked as enrolled in the UI
4. Student can view enrolled courses in My Courses
 - Given I am logged in as a student and enrolled in one or more courses
 - When I navigate to the **My Courses** page
 - Then I can see all courses I am currently enrolled in
 - And the newly enrolled course appears in that list
5. Student gains access to course materials after enrolling
 - Given I am logged in as a student and have enrolled in a course
 - When I open that course's detail or materials page
 - Then I am allowed to access the course materials
 - And the content is no longer locked or restricted
6. Student cannot enroll in the same course twice
 - Given I am logged in as a student and already enrolled in a course
 - When I attempt to enroll in that same course again
 - Then the system prevents a duplicate enrollment
 - And I see a clear message such as **Already enrolled**
 - And no duplicate enrollment record is created
7. Non-enrolled student cannot access course materials

- Given I am logged in as a student but not enrolled in a course
- When I try to open that course's materials
- Then access is denied
- And I see a message indicating enrollment is required

Integration Tests [Interaction between Features]

1. **Register professor + student test**
Create one professor and one student via API/UI and confirm both persist in DB with correct role flags.
2. **Duplicate registration conflict test**
Try registering an existing email and verify HTTP 409 with the expected conflict message.
3. **Login/session integration test**
Login as each role and confirm `/api/auth/session` returns correct role, `userId`, and dashboard path.
4. **Logout/session invalidation test**
Logout, then call `/api/auth/session`; verify it returns unauthorized (401).
5. **Professor course creation success test**
As professor, create a course and confirm:
 - API returns 201
 - course appears in professor dashboard
 - row exists in courses table
6. **Course creation validation test**
Submit blank title or code; verify validation error (422) and no DB insert.
7. **Role authorization test (create)**
As a student, attempt `POST /api/courses`; verify 403.
8. **Ownership authorization test (update/delete)**
As a different professor (not owner), attempt `PATCH` and `DELETE` on another professor's course; verify 403.
9. **Owner edit integration test**
As a owning professor, update description, tags, material, dueDate, link; verify API response + dashboard + DB all reflect new values.
10. **Owner delete + cascade test**
Delete a course as owner and verify:
 - API returns 204
 - course disappears from dashboard immediately
 - related enrollments/reviews rows are removed (cascade behavior)
11. **Validation safety invariant test**
Submit an invalid course creation request and confirm:
 - API returns 422 validation error

- no new course is inserted
- total course count remains unchanged

12. **Multi-user data isolation test**

Create a course as Professor A, then log in as Professor B and confirm:

- Professor B cannot see Professor A's courses in B's dashboard/list
- user data separation is preserved

13. **Logout invalidation protection test**

Login, perform logout, then attempt a protected action with the same session and confirm:

- logout returns 204
- subsequent protected call returns 401

14. **Course search integration pipeline test**

Create multiple courses and verify search endpoint behavior:

- API returns 200
- results are filtered by title keyword
- only matching courses are returned
- non-matching courses are excluded

PLATFORM/SMOKE TEST (NON-FEATURE SPECIFIC)

1. contextLoads: verifies Spring application context boots successfully with test configuration.

Test Level	Scope & Requirement	Methodology (How will you do this?)
Unit Testing	At least 19 unit tests per core feature(Sprint 2). Total: 41 tests.	<i>We use JUnit, MockMvc, and Spring Boot Test to mock dependencies and verify logic in isolation.</i>
Integration Testing	14 tests total covering interactions between features.	<i>End-to-end validation of cross-feature API and session flows: auth, role/ownership authorization, course CRUD, DB persistence/cascade integrity, and multi-user isolation.</i>
Acceptance Testing	End-user testing for every user story.	<i>Team members will perform Manual Walkthroughs based on User Story criteria.</i>
Regression Testing	Unit + Integration tests executed on every push/PR to the main branch.	<i>We have configured a GitHub Actions CI pipeline to run all tests automatically.</i>
Mutation Testing	52 tests total	<i>Mutation testing will be conducted on backend service-layer logic by running PIT against the selected feature classes and then executing the related JUnit 5 test suites. PIT creates small changes in the compiled code, re-runs the tests, and reports whether those tests were strong enough to detect the injected fault.</i>

2.1.2 CI/CD Regression Workflow

Regression tests:

- All unit tests and integration tests are executed for each commit pushed to the main branch.
- *"Every time a push/pull request is created, our CI server triggers the Unit and Integration suites."*

3 Terms/Acronyms

TERM/ACRONYM	DEFINITION
API	Application Program Interface
AUT	Application Under Test
PIT	A mutation testing tool for Java that generates code mutants and evaluates whether the test suite detects them.
Mutant	A modified version of the original program created to simulate a small fault.
Killed Mutant	A mutant detected by one or more failing tests.
Survived Mutant	A mutant that does not cause any related test to fail.
Equivalent Mutant	A mutant that changes the code syntactically but not the observable behavior.
CI/CD	Continuous Integration / Continuous Delivery.